

Universidade de Caxias do Sul (UCS)
Área de Ciências Exatas e Tecnologia
Disciplina: Programação de Computadores I

Linguagem de Programação C

Conceitos Básicos

Prof. Leonardo Pellizzoni

Algoritmo

- Um algoritmo é uma lista ordenada de comandos que, quando executados, resultam em uma sequência de ações, que visam resolver um determinado problema.
- Ex:????
- Algoritmos podem ser descritos de diversas formas, gráficas ou textuais.
- Quando um algoritmo é escrito em uma **linguagem de programação**, é chamado de **programa**.
 - Quantas linguagens de programação você conhece?

Algoritmo

Para a construção de qualquer tipo de algoritmo, é necessário seguir estes passos:

1. Compreender completamente o problema a ser resolvido, destacando os pontos mais importantes e os objetos que o compõem.
2. Definir os dados de entrada, ou seja, quais dados serão fornecidos e quais objetos fazem parte desse cenário-problema.
3. Definir o processamento, ou seja, quais cálculos serão efetuados e quais as restrições para esses cálculos. O processamento é responsável pela transformação dos dados de entrada em dados de saída. Além disso, deve-se verificar quais objetos são responsáveis pelas atividades.

Algoritmo

Para a construção de qualquer tipo de algoritmo, é necessário seguir estes passos:

4. Definir os dados de saída, ou seja, quais dados serão gerados depois do processamento.
5. Construir o algoritmo utilizando um formalismo conhecido.
6. Testar o algoritmo realizando simulações.

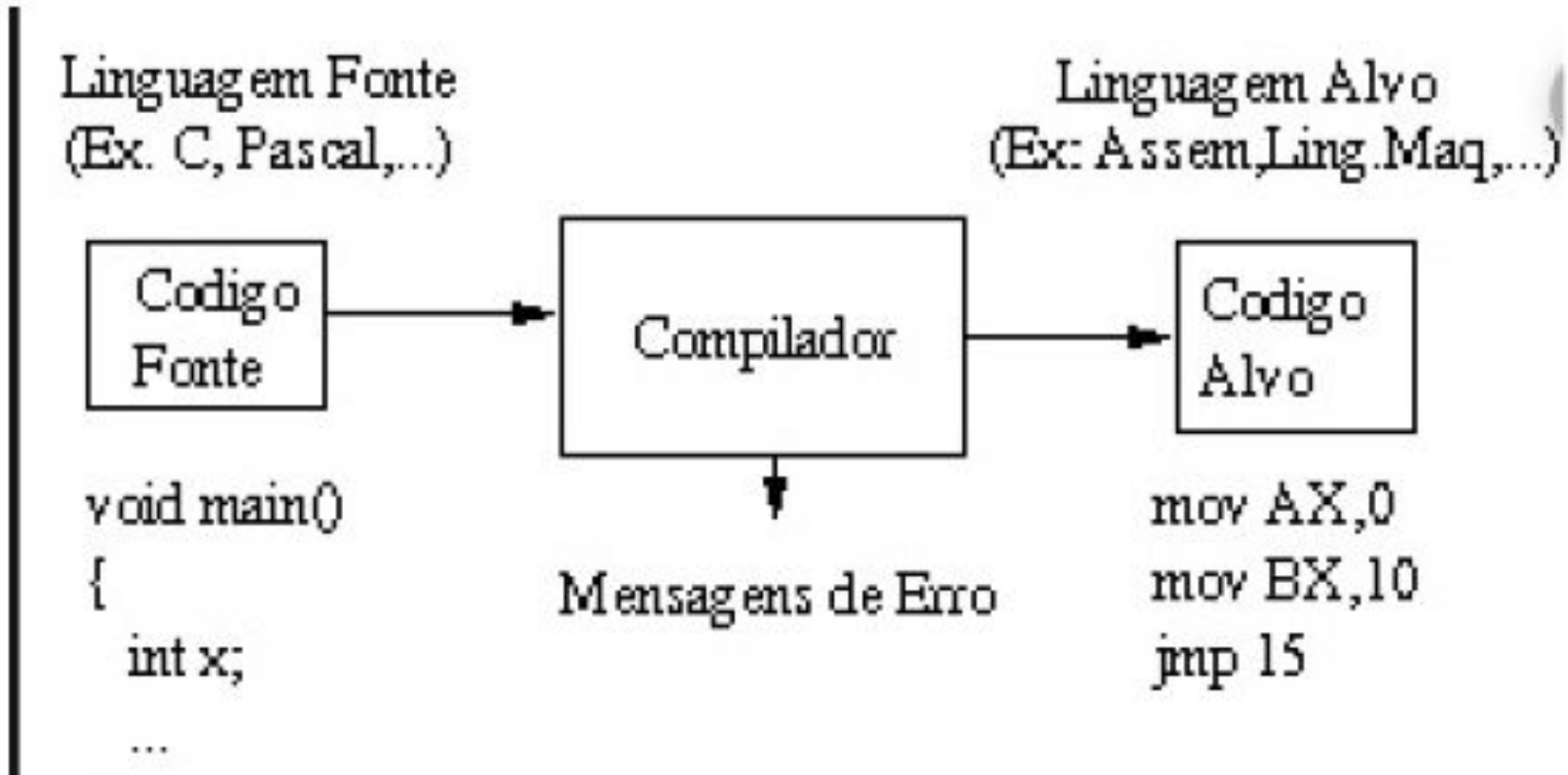
Linguagem C

- A linguagem de programação C foi desenvolvida em 1972 por Denis Ritchie, e ainda é a linguagem mais usada na programação de microcontroladores e na indústria.
- Como linguagem de uso geral, é a segunda linguagem mais utilizada no mundo, perdendo apenas para Python.

Compilação

- Para que um programa escrito em uma linguagem de programação (dito "alto nível") possa ser executado, ele deve ser traduzido para uma "linguagem de máquina"
- Essa função é executada por um programa chamado compilador, normalmente associado a um ambiente de desenvolvimento (IDE - Integrated Development Environment) que contém editor e recursos para execução do programa

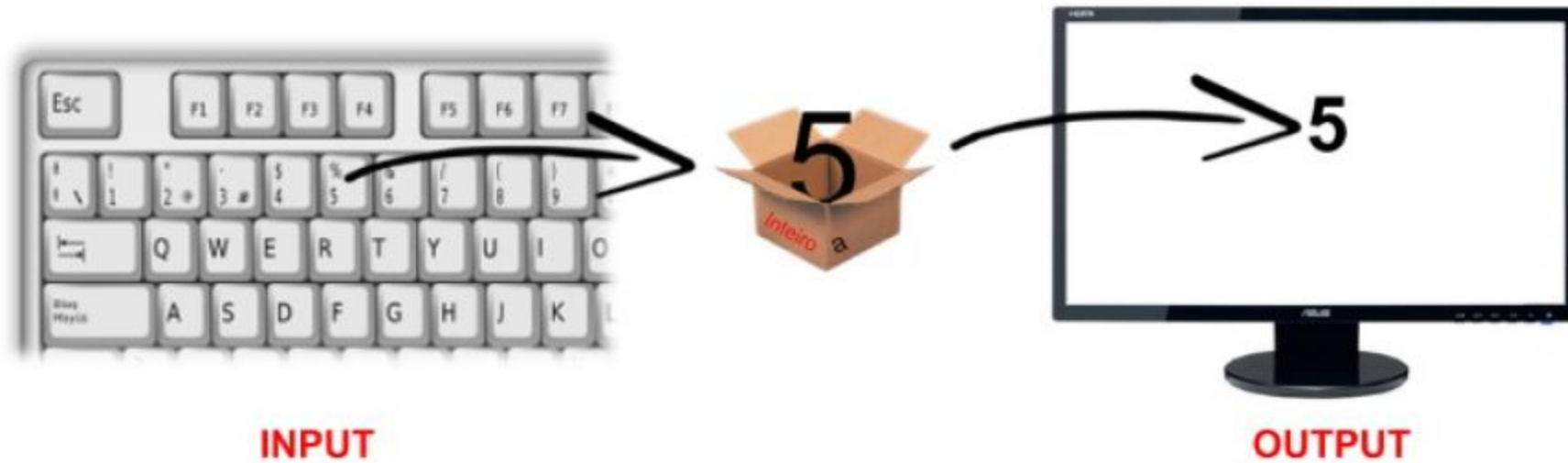
Compilação



Programas de Computador

- Programas de computador basicamente recebem dados, processam-nos e emitem resultados.
- Esses dados podem vir de várias fontes:
 - Teclado
 - Sensores
 - Arquivos
- O resultado pode ser enviada para vários lugares:
 - Monitor
 - Atuadores (ligar motores, acender luzes, emitir sons)

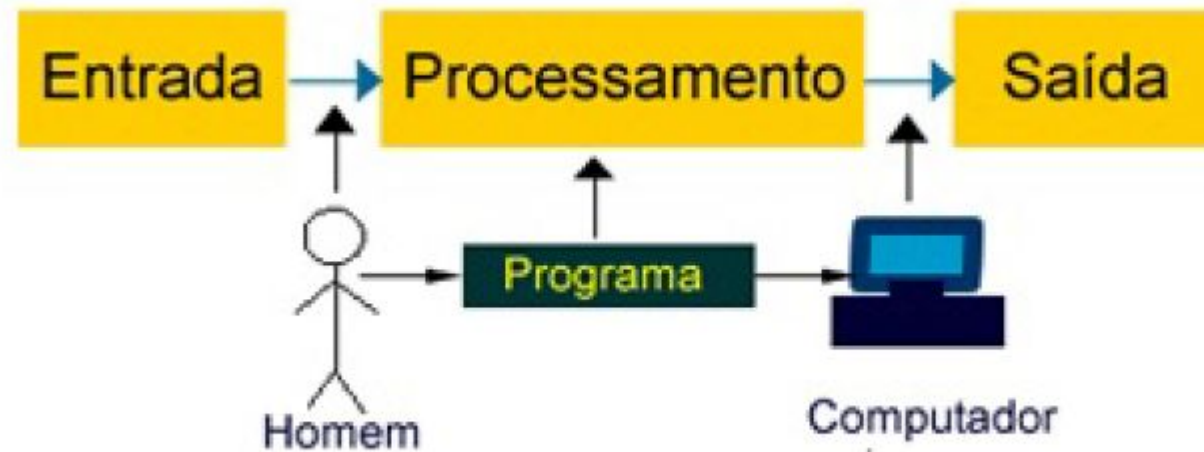
Programas de Computador



Operação de entrada de dados (input)

Programas de Computador

- Nessa disciplina, nossos programas receberão dados pelo teclado, digitados pelo usuário, e os resultados serão escritos na tela



Lembrando

Para a construção de qualquer tipo de algoritmo, é necessário seguir estes passos:

1. Compreender completamente o problema a ser resolvido, destacando os pontos mais importantes e os objetos que o compõem.
2. Definir os dados de entrada, ou seja, quais dados serão fornecidos e quais objetos fazem parte desse cenário-problema.
3. Definir o processamento, ou seja, quais cálculos serão efetuados e quais as restrições para esses cálculos. O processamento é responsável pela transformação dos dados de entrada em dados de saída. Além disso, deve-se verificar quais objetos são responsáveis pelas atividades.

Lembrando...

Para a construção de qualquer tipo de algoritmo, é necessário seguir estes passos:

4. Definir os dados de saída, ou seja, quais dados serão gerados depois do processamento.
5. Construir o algoritmo utilizando um formalismo conhecido.
6. Testar o algoritmo realizando simulações.

Exemplo

Suponha um programa para calcular o volume de um cilindro de raio r e altura h .

- Quais os dados de entrada?
- Que cálculos devem ser realizados?
- Qual a saída esperada? O que o programa deve escrever?
- O que seria um exemplo de entrada e saída esperada para a entrada?

Ferramenta <http://pythontutor.com/>

- Utilizaremos a ferramenta <http://pythontutor.com/> para poder acompanhar os testes que faremos a partir de agora!
- <http://pythontutor.com/> ! Start visualizing your code now
- Selecionar linguagem
- Editar código (p.ex. código abaixo)
- Visualize execution

Ferramenta <http://pythontutor.com/>

Write code in C (gcc 4.8, C11) EXPERIMENTAL ▼

```
1 int main() {  
2     int a=0;  
3     printf("%d\n",a);  
4     return 0;  
5 }
```

Keep this tool free for everyone by [making a small d](#)

Support our research by completing a [short user sur](#)

Visualize Execution

Ferramenta <http://pythontutor.com/>

- Ao clicar em "visualize execution" é feita uma simulação da execução do código (aparece uma mensagem pedindo para esperar 10 segundos) e é gerada uma sequência de frames sobre a qual é exibida a simulação.
- Eventualmente o servidor pode estar sobrecarregado, ou a duração da simulação excede o limite de frames permitido, e não será possível fazer a simulação.
- Como a exibição é feita após a simulação, não é possível interação com o usuário (ou seja, não há comandos de entrada (scanf))
- Dados de teste podem ser inseridos na declaração de variáveis.

Ferramenta <http://pythontutor.com/>

C (gcc 4.8, C11) **EXPERIMENTAL!**
see [known bugs](#) and report to philip@pgbovine.net

```
1 #include <stdio.h>
2 typedef struct nodo{int val;struct nodo *prox;} Tnodo;
3 int main() {
4 int m[10]={9,8,7,6,5,4,3,2,1,0},i,j,aux;
5 for (j=0;j<9;j++)
6 for (i=0;i<9;i++)
7     if (m[i]>m[i+1])
8     {
9         aux=m[i];
10        m[i]=m[i+1];
11        m[i+1]=aux;
12    }
13 return 0;
14 }
```

[Edit code](#)

→ line that has just executed

→ next line to execute

Click a line of code to set a breakpoint; use the Back and Forward buttons to jump there.



<< First < Back Step 179 of 318 Forward > Last >>

Stack

main									
array									
	0	1	2	3	4	5	6	7	8
m	int	int	int	int	int	int	int	int	int
	4	5	3	2	1	0	6	7	8
i	int								
	1								
j	int								
	4								
aux	int								
	5								

Ferramenta <http://pythontutor.com/>

- Os botões First e Last permitem ir até o primeiro e o último frame da simulação
- Os botões Back e Forward permitem avançar ou voltar um frame
- É possível setar pontos de parada no código para que, ao chegar no ponto selecionado, a execução seja pausada
 - Isso é feito clicando uma vez sobre a linha a ser inserido o break point. A linha cara em vermelho
 - Ao clicar em Forward, ao passar sobre o ponto de parada a simulação será interrompida
 - Ao clicar em Last os breakpoints serão ignorados
 - Para remover o breakpoint, basta clicar novamente sobre a linha

Linguagem C

- Bibliotecas
- Bloco do programa principal
- Variáveis
- Comando de Atribuição
- Declaração de variáveis
- Comentários
- Printf
- Scanf
- Comando de Atribuição

Bibliotecas

- A linguagem C possui várias bibliotecas (conjuntos de funções prontas)
- Bibliotecas para leitura e escrita, matemáticas, para manipulação de textos, etc
- Para usar funções de uma biblioteca, é necessário informar no início do programa quais bibliotecas serão utilizadas
- Isso é feito com a cláusula `#include <nome da biblioteca>`
- Exemplo:

`#include <stdio.h>`

- A biblioteca `stdio.h` possui as funções de leitura e escrita (`printf` e `scanf`)

Bloco do programa principal (Função *main*)

```
int main ( )
```

```
{  
}
```

- Um programa em C é formado por um conjunto de blocos (chamados funções). Um dos blocos é chamado de main e é por onde inicia a execução do programa. O formato de cada bloco é:

```
tipo_de_valor_retornado_pelo_bloco nome_da_função ( parâmetros )
```

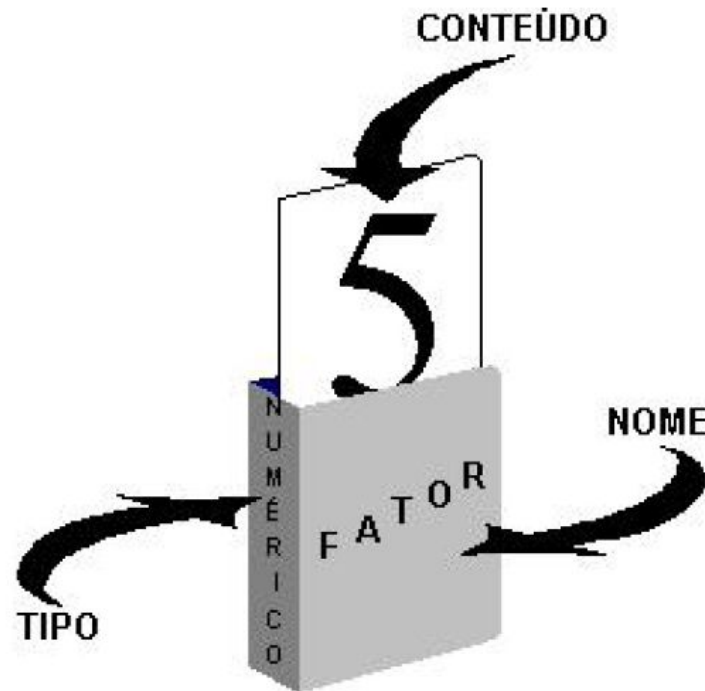
```
{  
}
```

Variáveis

- São os elementos de armazenamento de dados em um programa
- Uma variável corresponde fisicamente a uma posição de memória do computador
- Ela contém sempre um (1) valor
- Ao receber um novo valor, o valor anterior é substituído
- Uma variável é identificada por um nome, chamado identificador da variável
- Um identificador de variável deve iniciar por letra e só pode conter letras, dígitos e o caracter "sublinha". Não pode conter caracteres acentuados ou cedilha

Variáveis

- Não pode ser nenhuma palavra reservada da linguagem (exemplos: *printf*, *scanf*, *void*, *int*...)
- Uma variável pode ser vista como uma caixinha identificada por um nome e que pode armazenar valores de um tipo denido (por exemplo: inteiro ou real)



Comando de Atribuição

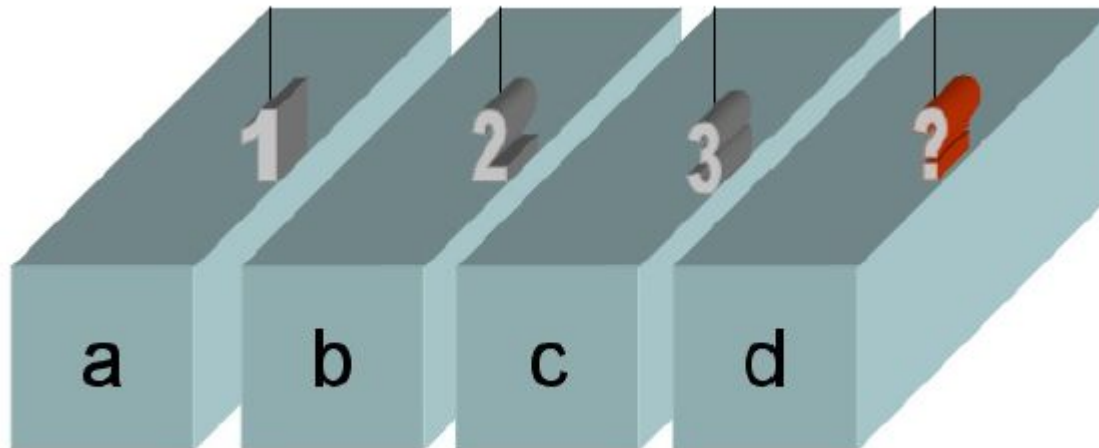
- O comando utilizado para alterar o valor de uma variável chama-se comando de atribuição e tem o formato:

variável = expressão; (leia-se "variável RECEBE expressão")

- Ex:
 - $a = 1;$
 - $a = a + 1;$
 - $a = a + b;$
- Na atribuição, quando uma variável recebe uma expressão, o valor da expressão é calculado e o resultado é atribuído a variável
- Ao utilizar constantes reais em expressões, o ponto (.) é utilizado como separador das casas decimais (ex: 3.14)

Comando de Atribuição

- Em um programa que contenha as variáveis a, b, c e d, após a sequência de atribuições a seguir:
 - `a = 1;`
 - `b = 2;`
 - `c = 3;`
- As variáveis terão os seguintes valores (o "?" em d significa que d ainda não tem um valor definido ou, em outras palavras, contém "lixo"):



Comando de Atribuição

Qual o valor das variáveis a e b ao final da execução do código abaixo?

a = 1;

b = 2;

a = a+b;

b = a+b;

a = a+b;

b = a+b;

a = a+b;

b = a+b;

escreva (a,b)

Declaração de Variáveis

- As declarações de variáveis em C tem o formato:
tipo <lista de variáveis>;
- Os principais tipos são **int** (variáveis inteiras) e **float** (variáveis reais). Ex:
int c1,b1,cont;
float peso, media;
- Os nomes de variáveis devem iniciar por letra e podem conter letras, dígitos e o caracter de sublinha (_).

Comentários

- Comentários podem ser escritos no código para melhorar sua clareza. Comentários são formados por qualquer sequência de caracteres delimitados por `/* ..... */`
- Os compiladores da linguagem C normalmente aceitam (apesar de não ser parte do C padrão) também comentários de linha, que iniciam por `//` e se estendem até o fim da linha
- Exemplo:
 - `int a,b,soma; /* Sou um comentário!!! */`
 - `printf("Digite um valor:"); // Eu também!!!`

Exemplo de Programa em C

```
# include <stdio .h>

// 04/08/2022

int main ( )
{
int lado , area ; /* declaração de variáveis */
printf (" Digite o comprimento do lado :");
scanf ("%d" ,& lado );
area = lado * lado ;
printf ("A area e %d\n",area );
}
```

Comando de escrita *printf()*

- A função **printf()** apresenta na tela valores de qualquer tipo de dado
- A função **printf()** está presente na biblioteca **stdio.h**
 - Não esqueça o **#include <stdio.h>**
- A função printf() tem como primeiro parâmetro a mensagem a ser mostrada na tela. Ex:

printf("Oi");

Comando de escrita *printf()*

- Se essa mensagem a ser mostrada na tela deve conter o valor de variáveis ou o resultado de expressões, as variáveis e expressões são listadas imediatamente após a mensagem. Ex:

a=3;

b=5;

printf("A soma de %d e %d vale %d",a,b,a+b);

Comando de escrita *printf()*

- O exemplo anterior mostrará a mensagem "A soma de 3 e 5 vale 8"
- Para exibir valores de variáveis e expressões, a mensagem deve conter uma indicação do local na mensagem onde serão inseridos os valores das variáveis e expressões
- Esses indicadores de posição tem o formato %d, se o valor a ser inserido for inteiro, e %f se o valor a ser inserido for float
- Os indicadores de posição devem ser em mesmo número dos elementos a serem inseridos, e na mesma ordem

Comando de escrita *printf()*

- **Um erro comum é utilizar o especificador %f para escrever valores inteiros ou %d para valores reais.** Alguns compiladores não identificam esse erro
- No especificador de formato pode-se especificar também o número de casas (total e após a vírgula), se o valor será alinhado à esquerda ou à direita e se o espaço previsto será preenchido ou não com 0s à esquerda.

Comando de escrita *printf()*

- Alguns exemplos são:
 - ***printf("%5d",x);*** - imprime o valor de x utilizando 5 espaços, alinhado à direita, preenchendo com espaços em branco à esquerda
 - ***printf("%05d",x);*** - imprime o valor de x utilizando 5 espaços, alinhado à direita, preenchendo com zeros à esquerda
 - ***printf("%5.3f",fx);*** - imprime o valor de fx utilizando 5 espaços no total, dos quais 3 após o ponto decimal, alinhado à direita, preenchendo com espaços em branco à esquerda
- Se o número de dígitos a ser mostrado exceder o tamanho definido para o campo, a informação de tamanho de campo é ignorada

Caracteres especiais

- Também podem ser utilizados caracteres de formatação na mensagem:
 - O principal caracter é o **\n** que causa uma quebra de linha no ponto onde foi inserido.
 - O caracter **\t** imprime um caracter de tabulação
 - O caracter **\r** posiciona o cursor no início da linha

Códigos de Formatação

- Códigos para formatação de impressão das expressões que devem ser passadas para o especificador de formato:
 - **%c** - caractere simples
 - **%d** ou **%i** - inteiro decimal com sinal
 - **%e** ou **%E** - notação científica
 - **%f** - ponto flutuante
 - **%g** ou **%G** - **%e** ou **%f** (o mais curto!)
 - **%o** - octal
 - **%s** - cadeia de caracteres
 - **%u** - inteiro decimal sem sinal
 - **%x** ou **%X** - hexadecimal
 - **%p** - ponteiro (endereço)
 - **%n** - ponteiro (inteiro)
 - **%%** - escreve o símbolo %

Comando de leitura *scanf()*

- O comando utilizado para receber dados do usuário através do teclado é o comando *scanf()*
- Os parâmetros do *scanf* são um especificador dos tipos dos valores que serão lidos, seguido da lista de variáveis que receberão os valores, sendo que o identificador de cada variável deve ser acompanhado de *&*

Comando de leitura *scanf()*

- Ex:
scanf ("%d%f", &a, &p);
- O exemplo acima recebe 2 valores do teclado, sendo o primeiro um valor inteiro e o segundo um valor float
- Os dois valores são armazenados respectivamente nas variáveis a e p
- Apenas os especificadores dos tipos (%f, %d, etc.) devem ir no primeiro parâmetro. Nada de espaços em branco ou outros caracteres. Coisas horríveis podem ocorrer se forem colocados outros caracteres.

Especificadores de formato

Especificador	Descrição	Caracteres lidos
i	inteiro	Qualquer quantidade de dígitos, opcionalmente precedidos por + ou -. Por padrão os dígitos são decimais (0 a 9), mas um prefixo 0 indica que são caracteres octais (0 a 7) e um prefixo 0x indica que são caracteres hexadecimais (0 a F)
d ou u	inteiro decimal	Qualquer quantidade de dígitos decimais (0-9), opcionalmente precedidos por sinal (+ ou -). d é para argumento com sinal, u para sem sinal
o	inteiro octal	Qualquer quantidade de dígitos octais (0-7), opcionalmente precedidos por sinal (+ ou -)
x	inteiro hexadecimal	Qualquer quantidade de dígitos hexadecimais (0-9,a-f,A-F), opcionalmente precedidos por 0x ou 0X, e opcionalmente precedidos por sinal (+ ou -)
f, e, g,a	float	Uma série de dígitos decimais, opcionalmente contendo um ponto decimal, opcionalmente precedido de sinal (+ ou -) e opcionalmente seguido por e ou E e um número decimal. Algumas implementações suportam floats em hexadecimal, quando precedidos por 0x ou 0X

Especificadores de formato

Especificador	Descrição	Caracteres lidos
c	caracter	Se um comprimento maior que 1 é especificado, a função lê a quantidade de caracteres especificada e armazena nas primeiras posições do vetor passado como parâmetro, sem acrescentar o \0 ao final
s	string de caracteres	Qualquer quantidade de caracteres exceto o espaço em branco. Um \0 é automaticamente adicionado ao final da sequência
p	pointer	Uma sequência de caracteres representando um endereço de memória. O formato específico depende do sistema usado, mas é o mesmo usado no %p no printf
[caracteres]	sequência de caracteres	Qualquer número de caracteres dentre os especificados entre os colchetes
[^ caracteres]	sequência de caracteres	Qualquer número de caracteres quaisquer, exceto os especificados entre os colchetes
%%	%	Dois % seguidos representam um %

Comando de atribuição

- Formato:
identificador_da_variável = expressão;
- Os principais operadores aritméticos da linguagem C são: adição (+), subtração (-), multiplicação (*), divisão (/)
- É importante, ao escrever uma expressão, considerar os tipos de operandos e o tipo do resultado da operação
- Os 4 operadores resultam int se ambos os operandos são int, caso contrário, resultam float.
 - **Atenção** para o caso do operador de divisão. Quanto resulta a expressão 3/2? E 3.0/2?

Comando de atribuição

- É importante também considerar a precedência (prioridade) dos operadores
- Os operadores $*$, $/$ e $\%$ tem a mesma prioridade entre si, que é maior que a prioridade dos operadores $+$ e $-$ (subtração).
- O operador $-$ também é utilizado como operador de troca de sinal, e nesse caso tem uma prioridade maior que todos os outros.

Principais operadores aritméticos

Operação	Operador	Prioridade
Troca de Sinal	-	1
Multiplicação	*	2
Divisão	/	2
Resto da Divisão Inteira	%	2
Soma	+	3
Subtração	-	3

Principais operadores aritméticos

- O operador de troca de sinal e o único operador de apenas um operando
- O operador de resto de divisão inteira só é aplicável a operandos inteiros
- Parênteses são utilizados para mudar a ordem de avaliação dos operadores em uma expressão
- A linguagem C faz conversão automática de tipos
 - Pode-se atribuir um int para uma variável float e vice-versa. Ao atribuir um float para um int, as casas decimais são truncadas